BAVote: Blockchain-Based Electronic Voting System With Privacy and Accountability

Ziyi Lin, Peng Jiang, Zijian Zhang, *Senior Member, IEEE*, and Liehuang Zhu, *Senior Member, IEEE*

***Abstract*—Blockchain-based electronic voting systems can achieve voter identity anonymity via cryptographic techniques such as ring signatures and blind signatures. However, fully hiding of voter information mitigates the capability of traceability. Previous mechanisms provide limited traceability, typically by preventing double-voting attacks while compromising anonymity. From the cryptographic point, threshold signature with private accountability seems to offer a balanced solution between privacy and accountability. If directly applying it into blockchain-based electronic voting systems, it needs to fix all voters and each verification has to pre-store all voters' public keys, incurring at least linear-size storage overhead and poor scalability. How to optimize the storage and scalability while guaranteeing both anonymity and traceability remains to be challenging. In this paper, we propose BAVote, an efficient and scalable blockchain-based electronic voting system with anonymity and traceability. It is built on top of a new threshold signature scheme named ConsATS that features a constant-size verification key. ConsATS compresses all voters' public keys into a single verification key, allowing an aggregated ballot to be verified without storing or processing per-voter public keys, thereby reducing on-chain storage overhead and improving scalability in BAVote. The aggregated signature serving as the ballot is encrypted in ConsATS, while BAVote further combines one-time addresses and a commit–reveal mechanism to protect intermediate on-chain data during voting. The corresponding tracing key enables authorized tracer to identify malicious voters during authorized audits. We implement a prototype of BAVote in both a local blockchain environment and the Ethereum Sepolia testnet. The experimental results show that the storage cost of verification keys in our system is reduced by more than 90% and the verification time is 7x faster compared to existing schemes.**

***Index Terms*—Blockchain, electronic voting systems, threshold signature, security and privacy.**

I. Introduction

BLOCKCHAIN-BASED electronic voting systems have emerged as a promising way for secure and fair voting [1], where each voter submits a vote and the relevant voting information is recorded on the blockchain. Only if the vote count exceeds a pre-set ratio, the proposal can pass. While the transparency of blockchain enhances the credibility of system, it conversely causes privacy leakage [2]. The vote content can be linked to a specific voter and the concrete content is accessible via on-chain records even though it adopts the secret ballot mechanism. From the perspective of voters, they are loath to expose their address on the blockchain and individual voting content. The linkability issue can typically be addressed through stealth address [3] and mixing technology [4]. Stealth address technology protects voter anonymity by generating disposable addresses for each vote and mixing technology enhances anonymity by scrambling transaction paths. Usually, content privacy leakage can be resisted through cryptographic techniques like blind signatures [5] and homomorphic encryption [6]. When using blind signatures, an authority can blindly sign the vote without learning anything about its content. When employing homomorphic encryption, the whole voting process can be launched over encrypted format due to the homomorphic feature, and thus there is no disclosure of the vote content.

Such a fully hiding mode incurs a failure of opening when the voting turns out to be malicious. For example, a malicious voter submits multiple votes via certain illegal means although each voter has only one vote, and controls the voting result. In this case, the voting system by applying the above privacy-preserving techniques cannot open the votes to find the malicious one. It is essential to endow the voting system with traceability, which enables the tracking of malicious voting behaviour once identified.

Threshold signature with private accountability (TAPS) provides a better-coupling way to achieve privacy and accountability. In such a signature mechanism, voters jointly generate a threshold signature such that the final signature hides the identities of the participating signers, while a designated tracing authority can run the tracing algorithm to identify misbehaving participants. Incorporating accountable threshold signatures into blockchain-based electronic voting systems still has limitations. Existing TAPS instantiation requires the verifier to maintain access to the public keys of all voters in order to verify any possible signing subset, resulting in $O(n)$ storage complexity. This linear dependency is problematic in blockchain settings, where on-chain storage is expensive and severely constrained. How to optimize the storage and scalability while guaranteeing both anonymity and traceability remains to be challenging.

In this paper, we propose an efficient and scalable blockchain-based electronic voting system with anonymity and traceability (BAVote). The proposed system is built on top of

Received 12 June 2025; revised 24 December 2025, 20 April 2026, and 25 May 2026; accepted 30 May 2026. Date of publication 5 June 2026; date of current version 12 June 2026. This work was supported by NSFC under Grant U2241213, Grant 62272038, and Grant 62272039. The associate editor coordinating the review of this article and approving it for publication was Dr. Arcangelo Castiglione. *(Corresponding author: Peng Jiang.)*

The authors are with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: 3120241291@bit.edu.cn; pengjiang@bit.edu.cn; zhangzijian@bit.edu.cn; liehuangz@bit.edu.cn).

Digital Object Identifier 10.1109/TIFS.2026.3700867

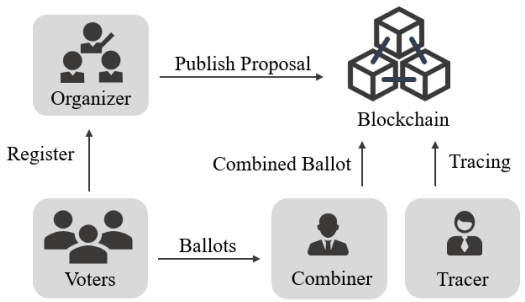


Fig. 1. System model.

a threshold signature scheme with private accountability and constant-size verification key (ConsATS). ConsATS follows the TAPS framework, while adopting an aggregation-based verification approach that enables all voters' public keys to be combined into a single verification key independent of the threshold value and the number of signers. Based on ConsATS, we design BAVote by integrating one-time addresses and a commit–reveal mechanism implemented through smart contracts. In this design, the aggregated signature serving as the final ballot is encrypted to protect the voting result, while the on-chain voting process is further safeguarded against privacy leakage from intermediate data. Traceability is achieved by enabling the decryption key to reveal the aggregated ballot, from which the voter set can be traced when required.

Our contributions can be summarized as follows.

- We present ConsATS, a threshold signature scheme with private accountability that supports anonymous signing quorum and a constant-size verification key. By adopting an aggregation approach, ConsATS eliminates the need for verifiers to store all participants' public keys, reducing the verification key storage from linear to constant size while preserving privacy and accountability.
- We design BAVote, a blockchain-based electronic voting system built on top of ConsATS. BAVote integrates a commit-reveal mechanism through smart contracts to ensure that both aggregated ballots encrypted in ConsATS and intermediate on-chain data do not leak voter identities, while enabling traceability.
- We implement and deploy a prototype of BAVote on both a local Ethereum network and the Sepolia testnet. Experimental results demonstrate that, for n = 50, our system reduces verification key storage by over 90% and achieves up to 7× faster verification compared with representative state-of-the-art schemes.

II. Blockchain-Based Electronic Voting System

A. System Model

The system model of the blockchain-based electronic voting system is shown in Fig. 1. The system consists of five main entities: the voting organizer, voters, combiner, tracer, and the blockchain network.

- Voting Organizer: The voting organizer is responsible for initiating the voting process. Before the voting starts, they initialize the voting information and deploy relevant smart contracts on the blockchain.
- Voters: A group of users granted voting rights. Voters are responsible for creating ballot information and casting their votes within the specified time.
- Combiner: The combiner is responsible for collecting all ballots from voters and verifying their validity. It then combines the valid ballots and uploads the combined result to the blockchain.
- Tracer: After the voting result is published, the tracer can trace the contributed voters from the combined votes and identify voters with abnormal voting behaviour.
- Blockchain: The blockchain serves as a storage medium for all voting-related data, including parameters, ballot information, voting results and so on.

Workflow. The system works as follows. Voters first register to obtain voting rights. The voting organizer initializes the voting information, including the proposal content, deadline, threshold, and other details, and publishes it on the blockchain to initiate the vote. Then voters construct their ballot information according to their preferences and upload it to the aggregator before the deadline. After the combiner verifies the validity of the ballots, it combines all valid ballots and uploads the combined ballot to the blockchain. The smart contract verifies the combined vote and announces the voting result. Based on the combined ballot, the tracer can trace the contributed voters, record any voters involved in abnormal voting behavior, and apply appropriate penalties.

B. System Design Goals

Voter anonymity: Voter anonymity ensures that contributed voters cannot be identified from the combined ballot recorded on-chain by participants other than the tracer who has been granted tracing rights.

Voter traceability: Traceability guarantees that the authorized tracer can trace the contributed voters based on the combined vote on the blockchain and hold those with abnormal voting behaviour accountable.

Efficiency optimization: The system reduces the storage overhead of verification keys and improves computational efficiency in the verification phase.

C. Threat Model

Adversary model. We consider two types of adversaries, honest-but-curious participants and malicious attackers. Honest-but-curious participants follow the protocol but attempt to infer private information during the voting process, such as the identity of contributed voters. Malicious attackers may control fewer than t participants and try to forge ballots to undermine the fairness of the voting system.

Based on the above two types of adversaries, we make the following assumptions. We assume that all adversaries are allowed to access the public parameters and the data stream transmitted over the public channel. The secret key is distributed through a secure channel and is confidentially maintained by the respective users. We assume that the majority of miners in the blockchain are honest and do not possess the ability to control more than 51% of the network nodes.

Trust assumptions. BAVote relies on designated combiner and tracer roles to support ballot aggregation and authorized auditing. Leakage or misuse of the tracing key may expose voter identities. However, the system's unforgeability and accountability guarantees remain preserved even if the combiner behaves maliciously or the tracer becomes compromised, thereby preventing forged ballots or false accusations against honest voters.

A fully decentralized realization could further distribute trust among multiple parties, but would introduce additional communication overhead, coordination complexity, and protocol latency. To maintain the efficiency, in this paper, we adopt the single-entity design as a practical trade-off between efficiency and decentralization.

III. ConsATS: Accountable Threshold Signature Scheme With Anonymous Signing Quorum and Constant-Size Verification Key

A. Definition of TAPS

A threshold signature scheme with private accountability [7] consists of the following five algorithms.

- $\mathsf{KeyGen}(1^\lambda, n, t) \to (pk, (sk_1, \cdots, sk_n), sk_c, sk_t)$: the key generation algorithm takes as input a security parameter λ, the number of signers n and threshold t, and outputs a public key pk, signers' secret keys $(sk_1, \cdots, sk_n)$, a combiner secret key sk_c and a tracing secret key sk_t.
- $\mathsf{Sign}(sk_i, m, C) \to \sigma_i$: the signing algorithm takes as input the signer's secret key sk_i, a message m and the signing quorum $C \subseteq [n]$. Then it computes the partial signature σ_i using sk_i.
- $\mathsf{Combine}(sk_c, m, C, \{\sigma_i\}_{i\in C}) \to \sigma$: the combining algorithm takes as input the combiner secret key sk_c, a message m, the signing quorum C and t signature shares $\{\sigma_i\}_{i\in C}$. It then outputs a TAPS signature σ.
- $\mathsf{Verify}(pk, m, \sigma) \to 0/1$: the verification algorithm verifies the signature σ on a message m with the public key pk and outputs the verification result.
- $\mathsf{Trace}(sk_t, m, \sigma) \to C/\mathsf{fail}$: the tracing algorithm takes as input the tracing secret key sk_t, the message m and the signature σ, and outputs a contributed signers set C or a message fail.
- For correctness we require that for all $1 \le t \le n$, all t-size sets $C \subseteq [n]$, all messages $m \in \mathcal{M}$, and for $(pk, (sk_1, \cdots, sk_n), sk_c, sk_t) \leftarrow \mathsf{KeyGen}(1^\lambda, n, t)$, the following two conditions hold:

$$\Pr[\mathsf{Verify}(pk, m, \mathsf{Combine}(sk_c, m, C, \{\sigma_i\})) = 1] = 1,$$
$$\Pr[\mathsf{Trace}(sk_t, m, \mathsf{Combine}(sk_c, m, C, \{\sigma_i\})) = C] = 1.$$

B. ConsATS Construction

The design of ConsATS is motivated by the inherent scalability limitations of the standard Threshold Accountable Private Signature framework. While TAPS provides a robust mechanism for private accountability, its Schnorr-based construction relies on the linear accumulation of public keys, resulting in an $O(n)$ storage overhead for the verifier. To enable deployment in resource-constrained blockchain environments, ConsATS leverages an accountable multsignature mechanism to achieve constant-size verification keys $vk = g_2^{\sum_{i=1}^n \alpha_i}$. However, this transition introduces a fundamental conflict between aggregation and privacy. In standard AMS schemes, the verification equation necessitates the inclusion of an identity-based product $\prod_{i\in C} H_1(i)$, which explicitly identifies the signer set C. To accommodate this setting, the zero-knowledge relation R_S is instantiated with respect to the aggregated verification equation, allowing the verifier to check correctness without learning the underlying signer set. A ConsATS scheme consists of the following five algorithms.

- $\mathsf{KeyGen}(1^\lambda, n, t) \to (pk, sk_c, sk_t, \{sk_i, pk_i\}_{i\in[n]}), vk, pkc)$: The key generation algorithm samples a bilinear group $\mathbb{PG} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, p)$ and two hash functions $H_0 : \mathcal{M} \to \mathbb{G}_1$, $H_1 : [n] \to \mathbb{G}_1$. It then generates a public key pk, a combiner secret key sk_c, a tracing secret key sk_t, the signers' public and secret key pairs $\{sk_i, pk_i\}_{i\in[n]}$, a public aggregation key pkc and a verification key vk.
  - Choose random $\alpha_i \leftarrow \mathbb{Z}_p$ and set $sk_i = \alpha_i$. For all $j \in [n]\backslash\{i\}$, compute $pk_{i,j} = H_1(j)^{\alpha_i}$ and $pk_i = (pk_{i,0}, \{pk_{i,j}\}_{j\in[n]\backslash\{i\}}) = (g_2^{\alpha_i}, \{pk_{i,j}\}_{j\in[n]\backslash\{i\}})$.
  - Encrypt the threshold t with pairing-based Elgamal encryption scheme. Generate $(sk_p, pk_p) \leftarrow \mathcal{PKE}.KeyGen$. Choose a random $\psi \in \mathbb{Z}_p$ and compute $z = e(g_1, g_2)$, $(T_0, T_1) = (g_2^\psi, pk_p^\psi z^t)$.
  - Generate $(sk_{cs}, pk_{cs}) \to \mathcal{SIG}.KeyGen$.
  - Generate $(sk_e, pk_t) \to \mathcal{PKE}.KeyGen$.
  - $pk = (pk_t, pk_p, pk_{cs}, T_0, T_1)$.
  - $sk_c = (pk, sk_{cs}, t, \psi)$.
  - $sk_t = (\{pk_i\}, pk, sk_e, t)$.
  - Compute verification key $vk = \prod_{i\in[n]} pk_{i,0} = \prod_{i\in[n]} g_2^{\alpha_i} = g_2^{\sum_{i=1}^n \alpha_i}$.
  - Compute public aggregation key $pkc = \{pkc_i\}_{i\in[n]}$ and $pkc_i = \prod_{j\in[n]\backslash\{i\}} pk_{j,i} = \prod_{j\in[n]\backslash\{i\}} H_1(i)^{\alpha_j}$.
- $\mathsf{Sign}(sk_i, m) \to \sigma_i$: Choose random $r_{i,m} \in \mathbb{Z}_p$ and compute $\sigma_i = (\sigma_{i,0}, \sigma_{i,1}) = (g_2^{r_{i,m}}, H_1(i)^{sk_i} H_0(m)^{r_{i,m}})$.
- $\mathsf{Combine}(pkc, sk_c, m, C, \{\sigma_j\}_{j\in C}) \to \sigma$: If $|C| = t$, the combiner generates the combined signature.
  - Combine $\sigma_0 = \prod_{j\in C} \sigma_{j,0} = g_2^{\sum_{j\in C} r_{j,m}}$ and $\sigma_1 = \prod_{j\in C}(\sigma_{j,1} \cdot pkc_j) = \prod_{j\in C} \left(H_1(j)^{\sum_{i=1}^n \alpha_i}\right) \cdot H_0(m)^{\sum_{j\in C} r_{j,m}}$, satisfying the equation $e(\sigma_1, g_2) = e(H_0(m), \sigma_0) \cdot e\left(\prod_{j\in C} H_1(j), vk\right)$.
  - Encrypt σ_1: choose a random $\rho \in \mathbb{Z}_p$, compute $c_0 = g_2^\rho, c_1 = pk_t^\rho \cdot e(\sigma_1, g_2)$ and $ct = (c_0, c_1)$.
  - Set $(b_1, \cdots, b_n) \in \{0, 1\}^n$, such that $b_i = 1$ iff $i \in C$. Then σ_0, σ_1 satisfy $e(\sigma_1, g_2) = e(H_0(m), \sigma_0) \cdot e\left(\prod_{i=1}^n H_1(j)^{b_j}, vk\right)$.
  - Generate a zero knowledge proof π of relation $\mathcal{R}_\mathcal{S}$.
  - Generate a signature $tg \leftarrow \mathcal{SIG}.Sign(sk_{cs}, (m, \sigma_0, ct, \pi))$ with combiner's key.
  - Output the signature $\sigma = (\sigma_0, ct, \pi, tg)$.
- $\mathsf{Verify}(vk, pk, m, \sigma) \to 1/\perp$: Output 1 if $\mathcal{SIG}.Verify\$pk_{cs}, \sigma, tg) = 1$ and π is a valid proof for relation $\mathcal{R}_\mathcal{S}$.
- $\mathsf{Trace}(sk_t, m, \sigma) \to C$:
  - the tracer parses $\sigma = (\sigma_0, ct, \pi, tg)$ and $ct = (c_0, c_1)$.

– It decrypts ct using decryption key sk_e including in tracing keysk_t.
– It finds a set $C \subseteq [n]$, where $|C| = t$ and

$$e(\sigma_1, g_2) = e(H_0(m), \sigma_0) \cdot e\left(\prod_{j\in C} H_1(j), \prod_{j\in[n]} pk_{j,0}\right).$$

This equality implies that σ is a valid signature. If the tracer finds such a signer set C, it outputs C. Otherwise, output fail.

The relation $\mathcal{R}_{\mathcal{S}}$.
Let $z = e(g_1, g_2) \in \mathbb{G}_T$ and $h_i = z^{u_i}$ with random $u_i \in \mathbb{Z}_p$.

$$\mathcal{R}_{\mathcal{S}} := \{(g_1, g_2, h_1, \cdots, h_n, z, vk, pk_p, pk_t, T_0, T_1, \alpha, \sigma_0, v_0, \cdots, v_n, ct = (c_0, c_1)); (\sigma_1, \rho, \psi, \gamma, b_1, \cdots, b_n, \phi_1, \cdots, \phi_n)\},$$

where $e(\sigma_1, g_2) = e(H_0(m), \sigma_0) \cdot e\left(\prod_{j\in[n]} H_1(j)^{b_j}, vk\right)$, $\$c_0, c_1) = (g_2^{\rho}, pk_t^{\rho} \cdot e(\sigma_1, g_2)), (T_0, T_1) = (g_2^{\psi}, pk_p^{\psi} z^t), v_0 = g_2^{\gamma}, v_i = h_i^{\gamma} z^{b_i}, \prod_{i=1}^n v_i^{\alpha^i(1-b_i)} = \prod_{i=1}^n h_i^{\phi_i}$.

1) The prover chooses random $\gamma \leftarrow \mathbb{Z}_p$ and commits to its bits $(b_1, \cdots, b_n) \in \{0,1\}^n$ as

$$(v_0 \leftarrow g_2^{\gamma}, v_1 \leftarrow h_1^{\gamma} z^{b_1}, \cdots, v_n \leftarrow h_n^{\gamma} z^{b_n}).$$

It sends $(v_0, v_1, \cdots, v_n)$ to the verifier.
2) The verifier samples a challenge $\alpha \leftarrow \mathbb{Z}_p$ and sends the challenge α to the prover.
3) The prover computes $\phi_i \leftarrow \alpha^i \gamma(1 - b_i)$ for $i \in [n]$.
4) Finally, the prover uses a Sigma protocol to prove the knowledge of a witness $(\sigma_1, \rho, \psi, \gamma, b_1, \cdots, b_n, \phi_1, \cdots, \phi_n)$ for the relation $\mathcal{R}_{\mathcal{S}}$.
   a) The prover chooses blinding factors

$$k_{\sigma_1}, k_{\rho}, k_{\gamma}, k_{\psi}, (k_{b1}, \cdots, k_{bn}), (k_{\phi 1}, \cdots, k_{\phi n}) \leftarrow \mathbb{Z}_p.$$

It then computes the following commitments.

$$S_1 \leftarrow e(g_1^{k_{\sigma_1}}, g_2) \cdot e\left(\prod_{j\in[n]} H_1(j)^{k_{bj}}, vk\right)^{-1},$$
$$S_{2a} \leftarrow g_2^{k_{\rho}}; \quad S_{2b} \leftarrow pk_t^{k_{\rho}} \cdot e(g_1^{k_{\sigma_1}}, g_2),$$
$$S_{3a} \leftarrow g_2^{k_{\psi}}; \quad S_{3b} \leftarrow pk_p^{k_{\psi}} \cdot z^{\sum_{i=1}^n k_{bi}},$$
$$S_{4a} \leftarrow g_2^{k_{\gamma}}; \quad \forall i \in \{1, \cdots, n\} : S_{4bi} \leftarrow h_i^{k_{\gamma}} z^{k_{bi}},$$
$$S_{4c} \leftarrow \prod_{i=1}^n v_i^{\alpha^i k_{bi}} h_i^{k_{\phi_i}}.$$

   b) The prover sends $(S_1, S_{2a}, S_{2b}, S_{3a}, S_{3b}, S_{4a}, \{S_{4b1}, \cdots, S_{4bn}\}, S_{4c})$ to the verifier.
   c) The verifier chooses a random challenge $\beta \leftarrow \mathbb{Z}_p$ and sends β to the prover.
   d) The prover computes

$$\widehat{\sigma}_1 \leftarrow \sigma_1^{\beta} g_1^{k_{\sigma_1}}, \widehat{\rho} \leftarrow \rho\beta + k_{\rho},$$
$$\widehat{\gamma} \leftarrow \gamma\beta + k_{\gamma}, \widehat{\psi} \leftarrow \psi\beta + k_{\psi},$$
$$\forall i \in \{1, \cdots, n\} : \widehat{b}_i \leftarrow b_i\beta + k_{bi}, \widehat{\phi}_i \leftarrow \phi_i\beta + k_{\phi i}.$$

It sends $\widehat{\sigma}_1, \widehat{\rho}, \widehat{\gamma}, \widehat{\psi}, \widehat{b}_i, \widehat{\phi}_i$ to the verifier.
   e) The verifier outputs "accept" if

$$S_1 \cdot e\left(\prod_{j\in[n]} H_1(j)^{\widehat{b}_j}, vk\right) \cdot e(H_0(m), \sigma_0)^{\beta} = e(\widehat{\sigma}_1, g_2),$$
$$S_{2a} \cdot c_0^{\beta} = g_2^{\widehat{\rho}}, S_{2b} \cdot c_1^{\beta} = pk_t^{\widehat{\rho}} \cdot e(\widehat{\sigma}_1, g_2),$$
$$S_{3a} \cdot T_0^{\beta} = g_2^{\widehat{\psi}}, S_{3b} \cdot T_1^{\beta} = pk_p^{\widehat{\psi}} \cdot z^{\sum_{i=1}^n \widehat{b}_i},$$
$$S_{4a} \cdot v_0^{\beta} = g_2^{\widehat{\gamma}}, \forall i \in \{1, \cdots, n\} : S_{4bi} \cdot v_i^{\beta} = h_i^{\widehat{\gamma}} z^{\widehat{b}_i},$$
$$S_{4c} \cdot \prod_{i=1}^n v_i^{\alpha^i \beta} = \prod_{i=1}^n v_i^{\alpha^i \widehat{b}_i} h_i^{\widehat{\phi}_i}.$$

Correctness. A valid signature $\sigma = (\sigma_0, ct, \pi, tg)$ generated by honest signers C ($|C| = t$) and a combiner will satisfy the verification equation because

$$\begin{aligned} e(\sigma_1, g_2) &= e\left(\prod_{j\in C} \left(H_1(j)^{\sum_{i=1}^n \alpha_i}\right) \cdot H_0(m)^{\sum_{j\in C} r_{j,m}}, g_2\right) \\ &= e\left(\prod_{j\in C} H_1(j), g_2^{\sum_{i=1}^n \alpha_i}\right) \cdot e\left(H_0(m), g_2^{\sum_{j\in C} r_{j,m}}\right) \\ &= e\left(\prod_{j\in C} H_1(j), vk\right) \cdot e(H_0(m), \sigma_0). \end{aligned}$$

And for any valid combined signature σ, the tracing algorithm Trace can correctly recover the signer set because

$$\begin{aligned} & e(H_0(m), \sigma_0) \cdot e\left(\prod_{j\in C} H_1(j), \prod_{j\in[n]} pk_{j,0}\right) \\ &= e\left(H_0(m), g_2^{\sum_{j\in C} r_{j,m}}\right) \cdot e\left(\prod_{j\in C} H_1(j), g_2^{\sum \alpha_j}\right) \\ &= e\left(\prod_{j\in C} H_1(j)^{\sum \alpha_j} \cdot H_0(m)^{\sum r_{j,m}}, g_2\right) \\ &= e(\sigma_1, g_2). \end{aligned}$$

Unforgeability and Accountability. The ConsATS scheme should satisfy the standard notion of unforgeability against a chosen message attack (EUF-CMA) and the scheme should be accountable. These simultaneous notions of unforgeability and accountability are referred to as *existential unforgeability under a chosen message attack with traceability* [7].

- Unforgeability: Unforgeability requires that an adversary controlling fewer than t signers, even when given the combiner secret key and the tracing secret key, cannot produce a valid combined signature that would be accepted by the verification algorithm.
- Accountability: Accountability requires that any valid combined signature can be correctly traced to the actual set of signers who contributed to the signature, and that an adversary cannot produce a valid signature that causes the tracing algorithm to fail or to blame an honest signer.

$(n, t, C, \mathsf{state}) \leftarrow \mathcal{A}_0(1^\lambda), \quad t \in [n],\ C \subseteq [n],$
$(pk, \{sk_i, pk_i\}_{i\in[n]}, sk_c, sk_t, vk, pkc) \leftarrow \mathsf{KeyGen}(1^\lambda, n, t),$
$(m', \sigma') \leftarrow \mathcal{A}_1^{\mathcal{O}(\cdot,\cdot)}(pk, \{sk_i\}_{i\in C}, sk_c, sk_t, vk, pkc, \mathsf{state})$
where $\mathcal{O}(C_j, m_j)$ returns the signature shares $\mathsf{Sign}(sk_i, m_j) i \in C_j$.
Winning condition:
Let $(C_1, m_1), (C_2, m_2), \cdots$ be $\mathcal{A}_1$'s queries to $\mathcal{O}$,
Let $C' \leftarrow \cup C_j$ for all queries where $m_j = m'$, let $C_t \leftarrow \mathsf{Trace}(sk_t, m', \sigma')$,
Output 1 if $\mathsf{Verify}(pk, vk, m', \sigma') = 1$ and either $C_t \nsubseteq C \cup C'$ or $C_t = \mathsf{fail}$.

Fig. 2. Unforgeability and accountability game.

$b \leftarrow \{0, 1\},$
$(n, t_0, t_1, \mathsf{state}) \leftarrow \mathcal{A}_0(1^\lambda),$ where $t_0, t_1 \in [n],$
$(pk, sk_c, sk_t, \{sk_i, pk_i\}_{i\in[n]}, vk, pkc) \leftarrow \mathsf{KeyGen}(1^\lambda, n, t_b),$
$b' \leftarrow \mathcal{A}_1^{\mathcal{O}_1(\cdot,\cdot,\cdot),\mathcal{O}_2(\cdot,\cdot)}(pk, \mathsf{state}),$
output $(b = b'),$
where $\mathcal{O}_1(C_0, C_1, m)$ returns $\sigma \leftarrow \mathsf{Combine}(pkc, sk_c, m, C_b, \{\mathsf{Sign}(sk_i, m)\}_{i\in C_b}),$
for $C_0, C_1 \subseteq [n]$ with $|C_0| = t_0, |C_1| = t_1,$
and where $\mathcal{O}_2(m, \sigma)$ returns $\mathsf{Trace}(sk_t, m, \sigma).$
Restriction: if σ is obtained from $\mathcal{O}_1(\cdot, \cdot, m),$ then $\mathcal{O}_2$ is never queried at $(m, \sigma).$

Fig. 3. Privacy game against the public.

$b \leftarrow \{0, 1\},$
$(n, t, \mathsf{state}) \leftarrow \mathcal{A}_0(1^\lambda),$ where $t \in [n],$
$(pk, sk_c, sk_t, \{sk_i, pk_i\}_{i\in[n]}, vk, pkc) \leftarrow \mathsf{KeyGen}(1^\lambda, n, t),$
$b' \leftarrow \mathcal{A}_1^{\mathcal{O}_1(\cdot,\cdot,\cdot),\mathcal{O}_2(\cdot,\cdot)}(pk, \{sk_i\}_{i\in[n]}, \mathsf{state}),$
output $(b = b'),$
where $\mathcal{O}_1(C_0, C_1, m)$ returns $\sigma \leftarrow \mathsf{Combine}(pkc, sk_c, m, C_b, \{\mathsf{Sign}(sk_i, m)\}_{i\in C_b}),$
for $C_0, C_1 \subseteq [n]$ with $|C_0| = |C_1| = t,$
and where $\mathcal{O}_2(m, \sigma)$ returns $\mathsf{Trace}(sk_t, m, \sigma).$
Restriction: if σ is obtained from $\mathcal{O}_1(\cdot, \cdot, m),$ then $\mathcal{O}_2$ is never queried at $(m, \sigma).$

Fig. 4. Privacy game against signers.

We formalize the unforgeability and accountability game in Fig. 2. Let $\mathbf{Adv}^{\mathrm{ua}}_{\mathcal{A},\mathcal{S}}(\lambda)$ be the probability that $\mathcal{A}$ wins the game in Fig. 2 against the ConsATS scheme.

During the game, the adversary is given the secret keys of a subset $C \subseteq [n]$ of signers, modeling an initial static corruption. In addition, the adversary obtains the combiner secret key sk_c and the tracing secret key sk_t. The adversary is also allowed to adaptively query a signing oracle $\mathcal{O}$, which on input (C_j, m_j) returns the corresponding partial signatures generated by signers in C_j on message m_j. The adversary's objective is to output a message-signature pair (m', σ') such that the signature is accepted by the verification algorithm, but the tracing algorithm either fails or outputs a signer set that includes at least one honest signer who did not contribute to the signature.

This game simultaneously captures both accountability and unforgeability. Even if the adversary corrupts at least t signers and possesses both sk_c and sk_t, it should be infeasible to produce a valid signature σ' for which $\mathsf{Trace}(sk_t, m', \sigma') = \mathsf{fail}$, or for which the tracing algorithm outputs a signer set that is not contained in the union of the corrupted signers and those queried to the signing oracle. Otherwise, the adversary succeeds in framing an honest signer or disabling the tracing mechanism, violating accountability. Suppose the adversary obtains fewer than the threshold number t of signature shares for the message m', that is, $|C \cup C'| < t$, where C' denotes the union of all signer sets queried to the signing oracle on message m'. If the adversary is able to produce a valid signature σ' such that $\mathsf{Verify}(vk, pk, m', \sigma') = 1$, then by the definition of the tracing algorithm, the tracing result C_t must satisfy $|C_t| \geq t$. Consequently, C_t cannot be contained in $C \cup C'$, implying that the tracing algorithm necessarily blames at least one honest signer. Hence, the adversary wins the game. Therefore, if no efficient adversary can win the above game, the scheme must be unforgeable.

Definition 1: A ConsATS scheme $\mathcal{S}$ is unforgeable and accountable if for all ppt adversaries $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$, the function $\mathbf{Adv}^{\mathrm{ua}}_{\mathcal{A},\mathcal{S}}(\lambda)$ is a negligible function of λ.

Privacy. We impose two privacy requirements.

- Privacy against the public. Privacy against the public requires that any external observer, who only has access to the public key pk and observes a sequence of message–signature pairs, learns nothing about the threshold t or the identities of the signers who generated the signatures.
- Privacy against signers. Privacy against signers requires that even a coalition of signers, who possess their own secret keys and observe a sequence of message-signature pairs, cannot determine which subset of signers contributed to the generation of a given valid signature.

We formalize these two privacy notions using two indistinguishability games, shown in Figures 3 and 4, respectively. Let $\mathbf{Adv}^{\mathrm{priv1}}_{\mathcal{A},\mathcal{S}}(\lambda)$ and $\mathbf{Adv}^{\mathrm{priv2}}_{\mathcal{A},\mathcal{S}}(\lambda)$ denote the probabilities that an adversary $\mathcal{A}$ wins the corresponding games against the ConsATS scheme.

In the privacy against the public game, the adversary selects two candidate thresholds t_0 and t_1 and adaptively chooses two signing quorums of corresponding sizes. The challenger generates system parameters using one of the two thresholds and answers signing queries using the hidden challenge bit. The adversary wins if it can distinguish which threshold was used, thereby learning information about the threshold or the signer set from the observed signatures.

In the privacy against signers game, the adversary is additionally given all signers' secret keys and may collude with all signers. The goal of the adversary is to distinguish which of two signing quorums of equal size was used to generate a challenge signature. The restriction that the tracing oracle cannot be queried on challenge signatures ensures that privacy is not trivially broken by invoking the tracing functionality.

Privacy against the public ensures that signatures do not leak information about the threshold or the signer set to external observers, while privacy against signers guarantees the privacy of signer set even in the presence of insider adversaries.

Definition 2: A ConsATS scheme is private if for all probabilistic polynomial time public adversaries $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$, the functions $\mathbf{Adv}^{\mathrm{priv1}}_{\mathcal{A},\mathcal{S}}(\lambda)$ and $\mathbf{Adv}^{\mathrm{priv2}}_{\mathcal{A},\mathcal{S}}(\lambda)$ are negligible functions.

C. Security Proof

Theorem 1: The proposed scheme is unforgeable and accountable, as in Definition 1, assuming the underlying ATS scheme is secure, and the non-interactive proof system (P, V) for $\mathcal{R}_{\mathcal{S}}$ is an argument of knowledge. For every adversary $\mathcal{A}$

We prove Theorem 1 via a sequence of games and a tight reduction to the underlying ATS scheme and the argument-of-knowledge property of the proof system (P, V).

Assumptions.

1) The underlying ATS scheme is $(t_{ATS}, q_{sig}, \epsilon_{ATS})$-secure.
2) The non-interactive proof system (P, V) is a computational argument of knowledge with knowledge error $\kappa(\lambda)$ and tightness $q(\lambda)$.

Game 0. This is the original unforgeability and accountability game from Fig. 2. Let W_0 be the event that adversary $\mathcal{A}$ wins. Then

$$\mathbf{Adv}^{ua}_{\mathcal{A},\mathcal{S}}(\lambda) = \Pr[W_0].$$

Game 1. Game 1 strengthens the winning condition: $\mathcal{A}$ must output a forgery (m', σ') along with a witness $w = (\sigma_1, \rho, \psi, b_1, \ldots, b_n)$ such that

$$\mathcal{R}_{\mathcal{S}}((pk_p, pk_t, vk, T_0, T_1, \sigma'_0, ct'); w) = \text{true}.$$

Construct a Game 1 adversary $\mathcal{A}'$ as follows:

1) Run $\mathcal{A}$ and answer its queries honestly.
2) When $\mathcal{A}$ outputs (m', σ'_0, ct'), apply the extractor *Ext* for (P, V) to obtain a witness $w' = (\sigma'_1, \rho', \psi', b'_1, \ldots, b'_n)$.
3) If w' is valid, compute π', tg' using w' and sk_c to form $\sigma' = (\sigma'_0, ct', \pi', tg')$.
4) Output (m', σ') and w'.

By the properties of the extractor, we have a tightness bound:

$$\Pr[W_1] \geq \frac{\Pr[W_0] - \kappa(\lambda)}{q(\lambda)}.$$

Game 2. Construct an adversary $\mathcal{B}_{\text{ATS}}$ that breaks the underlying ATS scheme using $\mathcal{A}'$:

1) Run $\mathcal{A}_0$ and obtain $(n, t, C, \mathsf{state})$. Send (n, t, C) to the ATS challenger and receive $(pk_1, \ldots, pk_n)$ and $(sk_1, \ldots, sk_n)$.
2) Generate $(pk_{cs}, sk_{cs}) \leftarrow \mathcal{SIG}.\mathsf{KeyGen}(\lambda)$ and $sk_e \in \mathbb{Z}_p$.
3) Compute $(T_0, T_1) = (g_2^{\psi}, pk_p^{\psi} z^t)$ as in KeyGen.
4) Construct (pk, sk_c, sk_t, vk) accordingly.
5) Run $\mathcal{A}_1$ with $(pk, sk_c, sk_t, \{pk_i, sk_i\}_{i \in C})$, answering signing queries via the ATS signing oracle.
6) Eventually, $\mathcal{A}_1$ outputs a forgery $(m', \sigma') = (m', (\sigma'_0, ct', \pi', tg'))$ and witness $w = (\sigma_1, \rho, \psi, b_1, \ldots, b_n)$.
7) Let $C' \subseteq [n]$ be the set corresponding to $(b_1, \ldots, b_n)$. Output the ATS forgery $(m', (\sigma'_0, \sigma_1, C'))$.

If $\mathcal{A}'$ wins Game 1 with probability $\Pr[W_1]$, then $\mathcal{B}_{\text{ATS}}$ breaks the ATS scheme with probability

$$\mathbf{Adv}^{forg}_{\mathcal{B},\mathcal{ATS}}(\lambda) \geq \Pr[W_1].$$

The running time satisfies $t_{\mathcal{B}_{\text{ATS}}} \leq t_{\mathcal{A}} + \text{poly}(\lambda)$ and the number of oracle queries satisfies $q_{\mathcal{B}_{\text{ATS}}} \leq q_{\mathcal{A}}$.

Advantage Bound. Combining the steps, the advantage of $\mathcal{A}$ is bounded as

$$\mathbf{Adv}^{ua}_{\mathcal{A},\mathcal{S}}(\lambda) \leq \mathbf{Adv}^{forg}_{\mathcal{B},\mathcal{ATS}}(\lambda) \cdot q(\lambda) + \kappa(\lambda),$$

where $q(\lambda)$ is the tightness of the proof system and $\kappa(\lambda)$ is the knowledge error.

This completes the reduction proof of unforgeability and accountability for the proposed scheme.

Fig. 5. The proof of unforgeability and accountability.

that attacks the scheme, there exists an adversary $\mathcal{B}$ that runs in about the same time as $\mathcal{A}$ such that

$$\mathbf{Adv}^{ua}_{\mathcal{A},\mathcal{S}}(\lambda) \leq (\mathbf{Adv}^{forg}_{\mathcal{B},\mathcal{ATS}}(\lambda)) \cdot q(\lambda) + \kappa(\lambda),$$

where κ is the knowledge error of the proof system and q is the tightness of the proof system.

Proof: By progressively modifying the game conditions and analyzing the adversary's advantage at each step, we reduce the security of our scheme to the underlying ATS scheme. The details of the proof process are shown in Fig. 5. ■

Theorem 2: The proposed scheme is private against the public, as in Definition 2, assuming DBDH holds in $\mathbb{G}_T$, the non-interactive proof system (P, V) for $\mathcal{R}_{\mathcal{S}}$ is HVZK, and the signature scheme $\mathcal{SIG}$ is strongly unforgeable. Concretely, for every adversary $\mathcal{A}$ that attacks scheme $\mathcal{S}$ there exist adversaries $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$ that run in about the same time as $\mathcal{A}$ such that

$$\mathbf{Adv}^{priv1}_{\mathcal{A},\mathcal{S}}(\lambda) \leq 2(\mathbf{Adv}^{eufcma}_{\mathcal{B}_1,\mathcal{SIG}}(\lambda) + Q \cdot \mathbf{Adv}^{hvzk}_{\mathcal{B}_2,(P,V)}(\lambda) + (Q+1) \cdot \mathbf{Adv}^{DBDH}_{\mathcal{B}_3,\mathbb{G}_T}(\lambda)),$$

where Q is the number of signature queries from $\mathcal{A}$.

Proof of Privacy.
We prove Theorem 2 via a sequence of games and reductions to the underlying assumptions: the strong unforgeability of the signature scheme $\mathcal{SIG}$, the HVZK property of the non-interactive proof system (P, V), and the DBDH assumption in $\mathbb{G}_T$.
Game 0. This is the original privacy game against the public as defined in Fig. 3. Let W_0 be the event that the adversary $\mathcal{A}$ outputs $b' = b$. Then

$$\mathbf{Adv}^{priv1}_{\mathcal{A},\mathcal{S}}(\lambda) = |2\Pr[W_0] - 1|.$$

Game 1. Modify Game 0 so that all tracing oracle queries $\mathcal{O}_2(\cdot,\cdot)$ always fail. By the strong unforgeability of $\mathcal{SIG}$, the adversary's advantage changes negligibly. Let W_1 be the event that $b' = b$. Then there exists an EUF-CMA adversary $\mathcal{B}_1$ such that

$$|\Pr[W_1] - \Pr[W_0]| \leq \mathbf{Adv}^{eufcma}_{\mathcal{B}_1,\mathcal{SIG}}(\lambda).$$

Game 2. Modify Game 1 so that the signing oracle $\mathcal{O}_1(C_0, C_1, m)$ generates the proof π using the HVZK simulator Sim on the statement $(pk_p, pk_t, vk, T_0, T_1, \sigma_0, ct)$. By HVZK, the adversary cannot distinguish simulated from real proofs. Let W_2 be the event that $b' = b$. Then there exists an HVZK adversary $\mathcal{B}_2$ such that

$$|\Pr[W_2] - \Pr[W_1]| \leq Q \cdot \mathbf{Adv}^{hvzk}_{\mathcal{B}_2,(P,V)}(\lambda),$$

where Q is the number of signing queries issued by $\mathcal{A}$. The factor Q arises from the hybrid argument over $\mathcal{A}$'s queries.
Game 3. Modify Game 2 so that the pairing-based ElGamal ciphertext (T_0, T_1) in KeyGen is replaced with a random element in $\mathbb{G}_1 \times \mathbb{G}_T$. By the DBDH assumption, the adversary cannot distinguish Game 3 from Game 2. Let W_3 be the event that $b' = b$. There exists a DBDH adversary $\mathcal{B}_3$ such that

$$|\Pr[W_3] - \Pr[W_2]| \leq \mathbf{Adv}^{DBDH}_{\mathcal{B}_3,\mathbb{G}_T}(\lambda).$$

Game 4. Finally, modify Game 3 so that the ciphertext $ct = (c_0, c_1)$ in each signing oracle response is chosen uniformly at random from $\mathbb{G}_1 \times \mathbb{G}_T$, instead of encrypting σ_1. By DBDH and a hybrid argument over Q queries, the adversary's advantage remains negligible. Let W_4 be the event that $b' = b$. Then there exists a DBDH adversary $\mathcal{B}'_3$ such that

$$|\Pr[W_4] - \Pr[W_3]| \leq Q \cdot \mathbf{Adv}^{DBDH}_{\mathcal{B}'_3,\mathbb{G}_T}(\lambda).$$

Conclusion. In Game 4, all ciphertexts and proofs are independent of b. Therefore, $\Pr[W_4] = 1/2$ and the adversary has zero advantage. Combining the bounds from all games, we obtain

$$\mathbf{Adv}^{priv1}_{\mathcal{A},\mathcal{S}}(\lambda) \leq 2\Big(\mathbf{Adv}^{eufcma}_{\mathcal{B}_1,\mathcal{SIG}}(\lambda) + Q \cdot \mathbf{Adv}^{hvzk}_{\mathcal{B}_2,(P,V)}(\lambda) + (Q+1) \cdot \mathbf{Adv}^{DBDH}_{\mathcal{B}_3,\mathbb{G}_T}(\lambda)\Big),$$

which is fully quantified and shows tightness with respect to the underlying assumptions.
This completes the rigorous reduction proof of privacy against the public for the proposed scheme.

Fig. 6. The proof of privacy.

Proof: By constructing a sequence of games, we reduce the advantage of the adversary in breaking the scheme's privacy against the public to the strong unforgeability of the signature scheme, the DBDH assumption, and the HVZK property of the zero-knowledge proof system. The details of the proof process are shown in Fig. 6. ∎

The proof of privacy against signers is mostly the same as the proof of Theorem 2 and is omitted.

IV. BAVote System Design

A. BAVote Overview

When applying ConsATS to BAVote, although the final combined ballot does not expose the signing quorum, the intermediate results generated during the voting process, namely the partial signatures, may reveal voter information while being transmitted on the blockchain. To mitigate this risk, we introduce the following additional measures in BAVote.

- Local key generation. The voters generate their public and private keys locally in BAVote. They then use their public keys to authenticate with the organizer to obtain the right to vote before the voting begins. In case of malicious voting behavior, the organizer can revoke the voting rights of the corresponding voter.
- One-time address. If the voters cast their votes using fixed addresses, an attacker could infer their identities based on the voting behavior such as voting time or geographical location. To break the association between the identities of the voters and their addresses, the voters use a one-time address to cast their votes in BAVote.

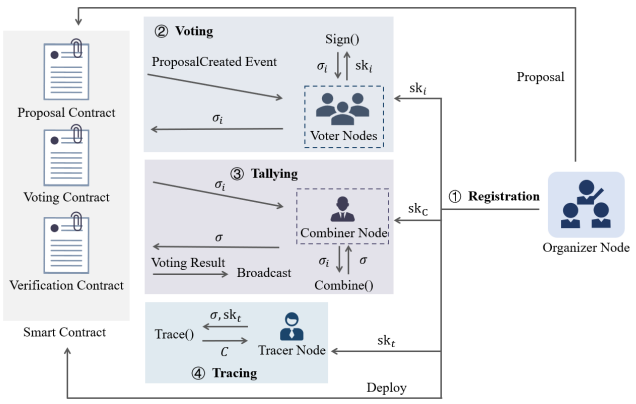


Fig. 7. BAVote.

Since the voters' public and private keys are generated locally, only the organizer can know the relationship between a voter's public key and the true identity. Meanwhile, the use of one-time addresses breaks the link between a voter's identity and address, ensuring that intermediate results in the voting process do not compromise voter privacy.

Fig. 7 illustrates the BAVote design. It consists of four phases: registration, voting, tallying, and tracing. Based on the ConsATS scheme, BAVote achieves voter anonymity and traceability, while the integration of smart contracts ensures that each phase of the voting process is executed in an orderly and verifiable manner by all participating nodes.

B. System Description

Registration Phase: In the registration phase, BAVote completes user registration and deploys the relevant smart contracts. The organizer first invokes the KeyGen algorithm to generate the cryptographic parameters required for the voting process, excluding the public and secret key pairs of individual voters. This process produces, among others, a combiner secret key and a tracing secret key, which are associated with the roles of the combiner and the tracer, respectively. The voters locally generate their public and secret key pairs and then authenticate with the organizer using their public keys to obtain voting eligibility. After authentication, each voter creates a one-time address for casting the vote.

The organizer then initializes the proposal information, including the proposal content, the voting threshold, and the voting deadlines. The voting threshold is published in encrypted form without revealing sensitive quorum information. Based on this information, the organizer deploys three smart contracts in BAVote: the proposal contract to publish proposals, the voting contract to manage vote submission and aggregation, and the verification contract to verify the correctness of the combined votes.

Proposal Publishing Phase. In BAVote, proposals are published by the organizer through the ProposalContract shown in Algorithm 1. To create a new proposal, the organizer invokes the createProposal function, which first verifies that the caller is indeed the organizer to ensure proper authorization. Upon successful verification, a new proposal is generated with a unique ID, the provided description, the creation timestamp, and an active status, and it is stored in the contract's proposal mapping. The contract then emits a ProposalCreated event, notifying all relevant network participants about the new proposal. After publication, voters can retrieve the details of the proposal by calling the getProposal function, enabling them to review the proposal content in preparation for voting.

Algorithm 1 Proposal Contract

```
1  Initialize proposalCount = 0 ;
2  Initialize proposals mapping as empty;
3  Function createProposal(description):
4      Verify caller is the organizer;
5      begin
6          Increment proposalCount;
7          Create new proposal: { id: current
             proposalCount, description: input description,
             createdAt: current block timestamp, active:
             true };
8          Store proposal in mapping;
9          Emit proposal created event;
10     end
11 end
12 Function getProposal(proposalId):
13     Verify proposalId is valid;
14     begin
15         Retrieve proposal from mapping;
16         return proposal id, description, creation time,
             status;
17     end
18 end
```

Voting Phase: In the voting phase, voters in BAVote cast their ballots for the published proposals through the voting contract shown in Algorithm 2. It follows a commit-reveal scheme. First, for each proposal, the organizer calls the initVoting function to initialize the voting window, setting the deadlines for the commit and reveal phases, and specifying the designated combiner. The contract then emits the VotingInitialized event, indicating that voting for the proposal has started.

During the commit phase, each voter calls commitVote using a one-time address to submit the hash of their voting intention, computed as $H(choice_i \| nonce_i)$, which locks the voting intention on-chain while preserving privacy. The contract records the commitment and emits the CommitSubmitted event. After the commit deadline is reached, the system enters the reveal phase. Each voter locally generates their partial signature by calling the Sign algorithm off-chain and then calls revealVote to submit their actual vote choice, the randomness nonce, and the partial signature σ_i to the contract. The contract verifies that the hash of the revealed choice and nonce matches the previously submitted commitment. It then marks the vote as revealed, records the submitted partial signature, and emits the VoteRevealed event.

Tallying Phase: After the reveal phase is completed, the designated combiner proceeds with the tallying phase. The

Algorithm 2 Voting Contract

```
Function initVoting(proposalId, commitPeriod, revealPeriod, combinerAddress):
    Verify caller is organizer from ProposalContract;
    Verify proposal exists and is active;
    begin
        commitDeadline ← currentTime + commitPeriod;
        revealDeadline ← currentTime + commitPeriod + revealPeriod;
        windows[proposalId] ← new Window: { commitDeadline, revealDeadline, combinerAddress, initialized: true };
        Emit VotingInitialized event;
    end
end
Function commitVote(proposalId, commitmentHash):
    Verify voting initialized for proposalId;
    Verify caller is registered voter;
    Verify within commit phase;
    begin
        commitments[proposalId][caller] ← commitmentHash;
        Emit CommitSubmitted event;
    end
end
Function revealVote(proposalId, choice, nonce, partialSignature):
    Verify within reveal phase;
    begin
        expectedHash ← hash(choice, nonce);
        Verify commitments[proposalId][caller] = expectedHash;
        revealed[proposalId][caller] ← true;
        if partialSignature is not empty then
            partialSig[proposalId][caller] ← partialSignature;
        end
        Emit VoteRevealed event;
    end
end
Function submitCombinedSignature(proposalId, combinedSignature):
    Verify caller is designated combiner for proposalId;
    Verify reveal phase completed;
    begin
        combinedSig[proposalId] ← combinedSignature;
        Emit CombinedSignatureSubmitted event;
    end
end
```

Algorithm 3 Verification Contract

```
Function confirmVerification(proposalId, accepted):
    verification ← verifications[proposalId];
    Verify verification.finalized = false;
    begin
        verification.exists ← true;
        verification.finalized ← true;
        verification.accepted ← accepted;
        Emit VerificationConfirmed event;
    end
end
Function getCombinedSignature(proposalId):
    combinedSig ← votingContract.combinedSig(proposalId);
    return combinedSig;
end
Function getVerificationStatus(proposalId):
    verification ← verifications[proposalId];
    if verification.exists = false then
        return "Not verified";
    end
    else
        if verification.accepted = true then
            return "Verified and accepted";
        end
        else
            return "Verified and rejected";
        end
    end
end
```

combiner collects all valid partial signatures submitted by voters during the reveal phase. Using these collected partial signatures as input, the combiner performs aggregation by invoking the Combine algorithm, which produces the combined signature $\sigma = (\sigma_0, ct, \pi, tg)$ for the proposal. Once the combined signature is generated, the combiner calls the submitCombinedSignature function on the VotingContract to upload the aggregated signature to the blockchain. The contract records the combined signature and emits the CombinedSignatureSubmitted event.

After the combiner submits the aggregated signature, the verification phase verifies the correctness of the combined vote. The organizer first calls getCombinedSignature function on the verification contract shown in Algorithm 3 to retrieve the aggregated signature from the VotingContract. Then, the organizer performs an off-chain verification by invoking the Verify algorithm, which checks the validity of the combined signature against the verification key and the proposal information. If the combined vote passes verification, it indicates that the number of voters supporting the proposal exceeds the threshold, leading to the proposal's approval.

Tracing Phase: In the tracing phase, BAVote supports vote traceability, enabling the authorized tracer to trace the voting result to the contributed voters.

If the voting results are contested, the tracer uses the on-chain record to locate the combined vote corresponding to the voting outcome. At this point, the tracing private key is made available to the tracer under predefined auditing conditions, as specified in the system design. Using this tracing private key,

TABLE I
COMPARISON OF KEY SIZE

Scheme	Signature Size	VK Size	VK Storage Complexity
TAPS [7]	$(n+4)\|\mathbb{G}\| + (2n+5)\|\mathbb{Z}_p\|$	$n \cdot \|\mathbb{G}\|$	$O(n)$
ConsATS (Ours)	$3\|\mathbb{G}_2\| + (2n+1)\|\mathbb{G}_T\| + (2n+2)\|\mathbb{Z}_p\|$	$\|\mathbb{G}_2\|$	$O(1)$

the tracer invokes the Trace algorithm to identify the set of contributed voters and to check for anomalous voting behavior. If any anomalous votes are detected, the voting organizer can hold the corresponding users responsible for the anomalous votes and invalidate the voting result.

C. System Analysis

1) Voter Anonymity: BAVote inherits the privacy property from ConsATS, which prevents external parties from identifying which voters contributed to an approved proposal from the aggregated ballot recorded on-chain. To further reduce privacy leakage during the voting process, BAVote employs one-time addresses to reduce linkage between voter identities and their on-chain voting activities. In addition, the commit-reveal mechanism prevents vote choices from being disclosed before the reveal phase, thereby mitigating premature inference of voting outcomes during the voting period.

2) Voter Traceability: BAVote supports voter traceability through the accountability property of ConsATS. The traceability is restricted to the designated tracer holding the tracing secret key. The tracer can decrypt the combined ballot and recover the set of voters contributing to the voting outcome by testing subsets of size t. It enables authorized tracing of anomalous voting behavior when disputes or malicious voting activities are identified.

3) Storage and Computational Overhead: As shown in Table I, ConsATS requires a single group element as the verification key, resulting in $O(1)$ verification key size and storage complexity, independent of the number of voters n. In contrast, TAPS require verification keys that scale linearly with n.

The on-chain storage overhead of BAVote is categorized into fixed infrastructure metadata and linear voter-related costs, which we analyze based on the PyPBC configuration ($qbits = 128, rbits = 256$). The fixed cost per proposal includes approximately 384 bytes of metadata for descriptors and voting windows, alongside a constant-size verification key $vk = g_2^{\sum \alpha_i}$, which is a single element in $\mathbb{G}_2$ requiring only 64 bytes. This property of the public verification infrastructure reduces the storage burden on the contract compared to traditional schemes that require storing n individual public keys. In contrast, the voter-specific storage grows linearly with n, where each voter contributes a 32 bytes commitment and a 96 bytes partial signature $\sigma_i = (\sigma_{i,0}, \sigma_{i,1}) \in \mathbb{G}_2 \times \mathbb{G}_1$, totaling $n \cdot (|\mathbb{G}_1| + |\mathbb{G}_2| + 32)$ bytes. Furthermore, the final combined signature $\sigma = (\sigma_0, ct, \pi, tg)$ introduces an $O(n)$ storage component due to the Zero-Knowledge Proof π. While $\sigma_0 \in \mathbb{G}_2$, $ct \in \mathbb{G}_2 \times \mathbb{G}_T$, and tg represent the fixed part of the signature totaling $2|\mathbb{G}_2| + |\mathbb{G}_T| + 64$ bytes, the proof π includes n additional elements in $\mathbb{G}_T$ (for S_{4bi}) and $2n$ scalars in $\mathbb{Z}_p$ (for $\widehat{b}_i, \widehat{\phi}_i$), resulting in an incremental cost of approximately 832 bytes per voter. The total storage per proposal with n voters is roughly $1024 + 960n$ bytes.

The on-chain computational cost is dominated by simple operations, as all complex cryptographic operations such as partial signature generation, signature aggregation, encryption, and zero-knowledge proof generation are performed off-chain by the voters and the combiner. On-chain, during the reveal phase, the main computation is a hash of the vote and nonce, which involves computing a 32 bytes hash and updating the mapping for each voter.

4) Decentralization of Combiner and Tracer: In our construction, the tracer and the combiner are modeled as single entities to support an efficient realization of anonymity and traceability under practical deployment constraints. While it is possible to decentralize the tracer and combiner by distributing the tracing secret key and combiner secret key among multiple parties, such a fully decentralized variant would inevitably introduce additional rounds of interaction, higher coordination complexity, and more involved state management. These factors directly increase protocol latency as well as on-chain and off-chain communication costs, which may not provide proportional benefits in our targeted setting. Therefore, we adopt single-entity roles as a design trade-off that aligns with our efficiency. Decentralized implementations based on Distributed Key Generation or Multi-Party Computation remain compatible with our overall framework and constitute a meaningful direction for future work.

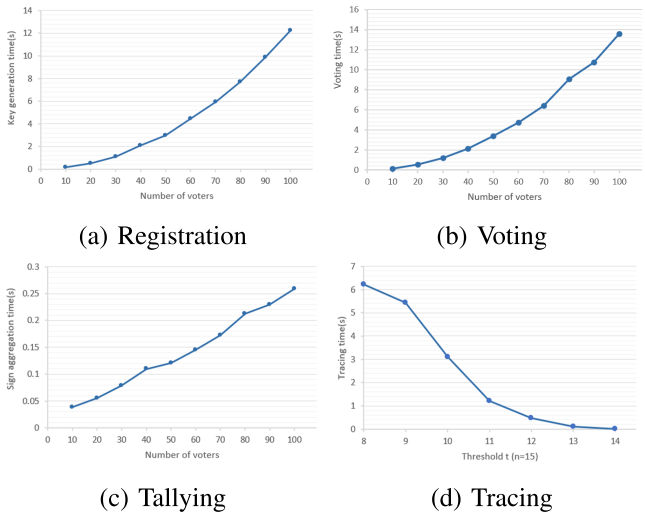
V. IMPLEMENTATION AND EVALUATION

A. Experimental Settings

We implement the voting system on a device equipped with an Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz, 16GB of RAM, running 64-bit Ubuntu 22.04.4. The five core algorithms of the scheme are implemented in Python. We use the PyPBC library to generate elliptic curve parameters for pairing-based cryptography, with reproducible parameter sets defined as qbits = 128 and rbits = 256 for the pairing groups. ECDSA is used as the digital signature scheme over the NIST P-256 curve, and SHA-256 is used as the hash function. For on-chain experiments, we deploy and evaluate our voting system on both a four-node local Ethereum network using Geth and the Sepolia Ethereum test network. We interact with the Sepolia Ethereum test network via Hardhat scripts configured to connect to an Infura RPC endpoint, submitting transactions on-chain to the Sepolia network.

B. Local Chain Performance Analysis

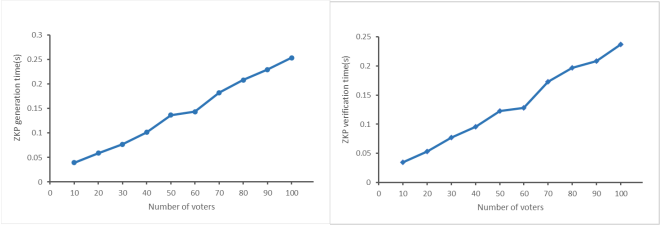
We deploy a four-node Ethereum network and simulate the voting process by having the organizer publish a proposal



(a) Registration (b) Voting

(c) Tallying (d) Tracing

Fig. 8. Execution time.



(a) ZKP generation time (b) ZKP verification time

Fig. 9. Execution time of ZKP.

on the chain. We vary the total number of voters n and the threshold t, and use the average running time of each phase as a measure of the efficiency of our system in the experiment. We also analyze the impact of the variation in message size on the voting process. Then we test the gas cost of deploying the smart contracts.

1) Scalability With Respect to the Number of Voters: We first evaluate the scalability of the system as the number of voters n increases. We vary n from 10 to 100 and set the threshold to $t = 0.7n$. In the registration phase, the dominant cost arises from executing the KeyGen algorithm. As shown in Fig. 8a, the registration time grows with n, reaching approximately 3 seconds when $n = 50$. During the voting phase, each voter executes the Sign algorithm. Consequently, the total voting time increases linearly with the number of voters, with an average execution time of 3.4 seconds at $n = 50$ (Fig. 8b). In the tallying phase, we measure the execution time of the Combine algorithm. As illustrated in Fig. 8c, the aggregation time is roughly proportional to the number of participating voters and averages 0.12 seconds when $n = 50$. For the tracing phase, we fix $n = 15$ and vary the threshold t from 8 to 14. As shown in Fig. 8d, the tracing time decreases as t increases, since fewer candidate signer sets need to be examined. The average tracing time is approximately 1.2 seconds at $t = 11$.

To evaluate the computational overhead introduced by the zero-knowledge proof, we measured the ZKP generation time in the Combine algorithm and the corresponding verification time in Verify for different numbers of voters. The experiment varied n from 10 to 100 in increments of 10, and for each configuration we computed the average latency over multiple runs. The result is shown in Fig. 12. The ZKP generation time increases from approximately 0.0396s at n = 10 to 0.2538s at n = 100, while the verification time grows from 0.0346s to 0.2369s over the same range. Both curves exhibit an approximately linear relationship with n, indicating that the proof system's computational cost scales proportionally with the number of partial signatures involved.

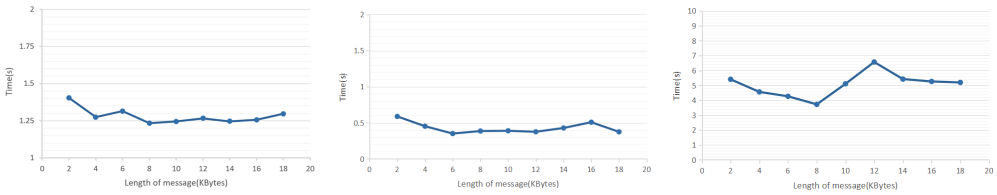
2) Impact of Message Size: We further evaluate the impact of message size on the execution time of different protocol phases. The message size m ranges from 2 KB to 18 KB, with the number of voters fixed at $n = 50$ and the threshold set to $t = 40$. As shown in Fig. 10, the vote generation time per voter remains stable at approximately 1.28 seconds. The execution times for the tallying and tracing phases are around 0.43 s and 5.06 s, respectively. Since the signing algorithm hashes the message prior to signing, the overall performance is only marginally affected by the message size.

3) Verification Time and Storage Cost Comparison: We compare the verification performance and storage cost of our scheme with the TAPS scheme. As shown in Fig. 11, the verification time of both schemes increases linearly with the number of voters. However, our scheme exhibits a significantly smaller constant factor. When $n = 50$, the average verification time of our scheme is approximately 0.11 seconds, whereas TAPS requires 0.87 seconds, yielding more than a $7\times$ speedup. In terms of storage cost of verification key, TAPS requires a verification key of size $n|\mathbb{G}|$, as the verifier must maintain public keys associated with all voters. In contrast, ConsATS only requires a verification key of size $|\mathbb{G}_2|$, thereby reducing the on-chain storage overhead.

4) On-Chain Performance Evaluation: We test the gas cost of deploying the smart contracts and the transaction latency as shown in Fig. 12a. The proposal contract, which publishes proposal information, costs 764775 gas. The voting contract uploads partial signatures from voters and costs 2330599 gas. For verifying the correctness of the signatures, the verification contract costs 667811 gas. As shown in Fig. 12b, the bars represent the transaction gas cost, while the line represents the transaction latency. Each voter's commit transaction consumes approximately 51366 gas, representing the per-vote cost during the commit phase. The average local transaction latency for a single commit transaction is around 0.018s. Under a setting with $n = 15$ voters, the cumulative latency for completing the entire commit phase across all voters is approximately 0.277s. During the reveal phase, each voter's reveal transaction incurs approximately 100855 gas as the per-vote transaction cost, with an average local transaction latency of around 0.017s per voter. The cumulative latency for completing the reveal phase is approximately 0.266s. The results show that the gas cost and transaction times of our system are within an acceptable range.

C. Evaluation on Sepolia Test Network

We deploy and evaluate our voting system on the Sepolia Ethereum test network, which provides a public-chain envi-



(a) Vote generation (b) Tallying (c) Tracing

Fig. 10. Execution time with varying m.

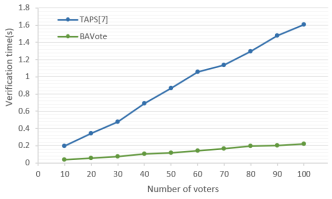
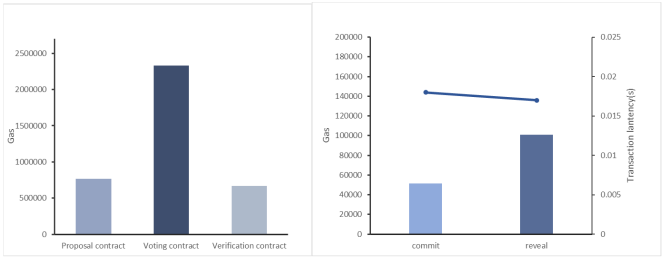
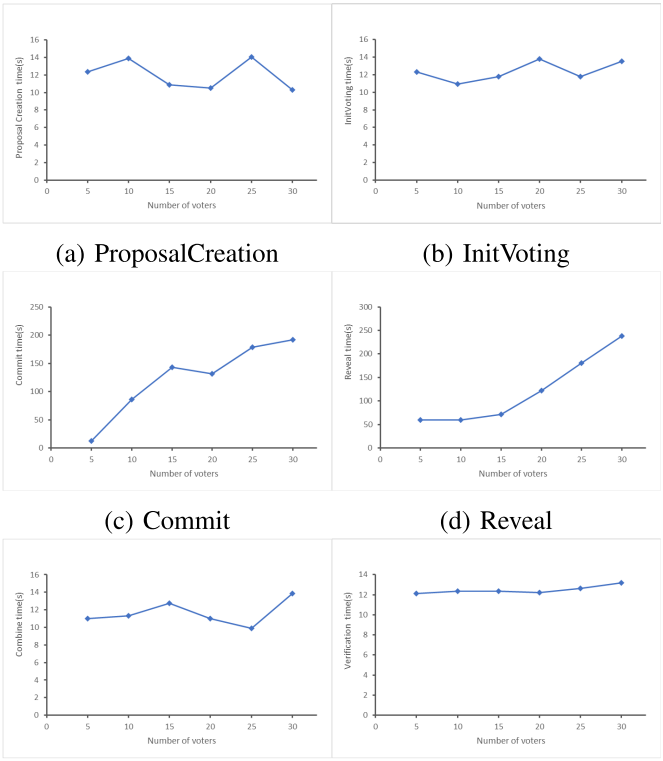


Fig. 11. Verification time.



(a) Deployment cost (b) Per-voter cost

Fig. 12. On-chain performance analysis.



(a) ProposalCreation (b) InitVoting

(c) Commit (d) Reveal

(e) Combine (f) Verification

Fig. 13. Transaction latency. Latencies for Commit and Reveal represent the cumulative elapsed time across all voters, reflecting the total cost of completing the voting process. The remaining phases correspond to single contract invocations executed once per voting round, and therefore reflect per-transaction latency.

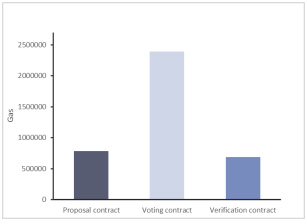


Fig. 14. Deployment cost.

ronment for measuring gas usage and transaction latency. We interact with the Sepolia Ethereum test network via Hardhat scripts configured to connect to an Infura RPC endpoint, submitting all transactions on-chain to the Sepolia network. We simulate the complete voting process with varying numbers of voters $n = 5, 10, 15, 20, 25, 30$, using a threshold $t = n/2$. For each configuration, we measure the on-chain performance of all key voting steps including CreateProposal, InitVoting, Commit, Reveal, Combine, and Verification. Specifically, for each step, we record both the transaction latency and the gas consumption, repeating each measurement multiple times and reporting the average values.

1) Transaction Latency Analysis: The recorded transaction latencies (in seconds) for each phase shown in Fig. 13 show that Commit and Reveal dominate the overall latency. The commit and reveal phases include all voters collectively, representing the total time for all voters. As the number of voters increases, Commit latency grows roughly linearly, reaching 191.69s for 30 voters, while Reveal latency also increases significantly, up to 238.13s. Other phases such as CreateProposal, InitVoting, Combine, and Verification remain relatively stable across different numbers of voters, with average latencies around 13s. This roughly corresponds to the time for a single transaction to be included in a block. This suggests that the operations in Commit and Reveal are the primary contributors to total voting latency.

2) Gas Consumption Analysis: The deployment of the three smart contracts on the Sepolia testnet was measured in terms of gas consumption in Fig 14. The proposal contract consumes 784595 gas, and the verification contract consumes

TABLE II
COMPARISON OF BAVOTE WITH RELATED VOTING SYSTEMS

Scheme	**BAVote**	[18]	[19]	[20]
Voter anonymity	Yes	Yes	Yes	Yes
Voter traceability	Full traceability	Double-voter tracing	Full traceability	Double-voter tracing
Tallying time	Milliseconds-level	Seconds-level	Seconds-level	-

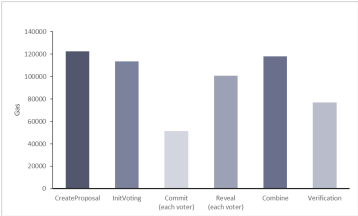


Fig. 15. Gas consumption. Gas costs for Commit and Reveal represent per-vote costs, as each voter submits an individual transaction in these phases. The remaining phases correspond to single contract invocations, and reflect per-transaction costs.

686099 gas. The voting contract is the most expensive, consuming 2395281 gas, likely due to the additional logic required to manage voter registration, commit–reveal voting, and interaction with the proposal contract.

The gas consumption for each step is shown in Fig 15. The CreateProposal phase consumes 122464 gas, representing the cost of storing the proposal on-chain. InitVoting consumes 113574 gas, which reflects initializing the voting state. The Commit phase requires 51366 gas per voter, as it only involves submitting a commitment hash. The Reveal phase is more expensive at 100855 gas per voter due to revealing the vote along with the associated proof. Combine consumes 118076 gas, Verification uses 76912 gas, since it mainly records the final verification result.

VI. RELATED WORK

A. Blockchain-Based Voting System

Blockchain technology has been increasingly applied to the field of electronic voting to build decentralized electronic voting systems in recent years. McCorry et al. [8] proposed a distributed self-tallying voting protocol based on Ethereum. Li et al. [9] introduced a self-tallying voting system in decentralized IoT based on blockchain. Yang et al. [10] introduced a blockchain-based self-tallying election protocol that supports score voting. In the electronic voting system, transparency and anonymity need to be considered simultaneously [11]. Meanwhile, the balance between anonymity and traceability in blockchain-based electronic voting systems has received increasing attention.

To prevent ballots from being traced back to the real identities of voters, several technologies such as blind signatures [12], ring signatures [13] and homomorphic encryption [14] have been proposed. Reference [15] used blind signatures to implement an anonymous e-voting system. Yu et al. [16] designed a verifiable electronic voting protocol based on smart contracts using ring signatures, and Wu et al. [6] combined homomorphic encryption to achieve privacy in voting systems. Li et al. [17] achieved voters' anonymity and ballot confidentiality using homomorphic time-lock puzzle and one-time ring signature. While these methods offer strong protection of voter identities, thereby achieving anonymity, they also present challenges such as enabling double voting. This requires a careful balance between anonymity and traceability.

In order to balance anonymity and traceability, Li et al. [18] proposed a voting system based on linkable group signatures and homomorphic time-lock puzzle. Miao et al. [19] used homomorphic time-lock encryption to preserve voting fairness and employed traceable ring signatures to enable traceability. However, both schemes take a long time to decrypt the ballots, which may affect the efficiency of the voting system. Li and Lai [20] introduced LaT-voting, which designed a voting protocol capable of tracing double voters.

The above works demonstrate that achieving anonymity and traceability simultaneously is feasible in blockchain-based voting systems. However, existing solutions are typically limited either by high tallying latency caused by expensive time-lock decryption or by restricted traceability that only supports double voter tracing. As summarized in Table II, while all compared systems achieve anonymity, none of them can simultaneously support full traceability and efficient tallying. BAVote aims to address these limitations and achieve a better balance among anonymity, traceability, and scalability.

B. Threshold Signatures

Threshold signatures [21] allow a group of n participants to jointly produce a valid signature on a message only if at least $t \leq n$ members participate in the signing process. Depending on whether the signature reveals the signing quorum, threshold signature schemes can be classified into private threshold signatures (PTS) [22], [23] and accountable threshold signatures (ATS) [24], [25]. In PTS schemes, the signature on a message m reveals nothing about the threshold t or the quorum of t signers, thus achieving complete privacy. In contrast, ATS schemes allow the signing quorum to be identified from the signature, ensuring complete accountability. To strike a balance between privacy and accountability, Boneh and Komlo [7] proposed the TAPS scheme, where both the threshold t and the signing quorum remain hidden from the public, but can be traced through a tracing secret key.

Moreover, most existing ATS constructions require verifier to store all n public keys of signers. To address this limitation, Das et al. [26] and Garg et al. [27] introduced multi-signature schemes with short verification keys by shifting the public key aggregation work from the verifier to the aggregator. Furthermore, Boneh et al. [28] proposed a novel multi-signature scheme that introduces an aggregation key algorithm to generate a constant-size verification key, further reducing the overhead for verifier.

The above works highlight the inherent trade-off between privacy, accountability, and efficiency in threshold signature

TABLE III
COMPARISON OF CONSATS WITH STATE-OF-THE-ART SCHEMES

	ConsATS	[7]	[26]	[27]	[28]
Verification key size	$1\|\mathbb{G}\|$	$n\|\mathbb{G}\|$	$7\|\mathbb{G}\|$	$3\|\mathbb{G}\|$	$1\|\mathbb{G}\|$
Privacy	Yes	Yes	No	No	No
Accountability	Yes	Yes	No	No	Yes
Storage complexity	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$

constructions. Schemes focused on privacy typically lack traceability, while current accountable schemes either suffer from linear verification key size or sacrifice signer privacy. Meanwhile, recent multi-signature schemes achieve short or constant-size verification keys, but they do not support accountability or quorum privacy. As summarized in Table III, although recent constructions successfully achieve constant-size verification keys and low storage complexity, they generally do not consider the coexistence of both privacy and accountability. This paper aims to address this gap by designing ConsATS, which achieves private accountability while enabling verification under a constant-size aggregated verification key.

VII. CONCLUSION

In this paper, we proposed ConsATS, a new threshold signature scheme with private accountability and constant-size storage cost for verification without revealing the identities of the signing quorum. Building on ConsATS, we designed BAVote, which integrates one-time addresses and a commit-reveal mechanism through smart contracts to prevent privacy leakage from intermediate on-chain data while supporting tracing when traceability is required. BAVote also aggregates all public keys into a constant-size verification key, which reduces the storage cost of verification keys and eliminates public key aggregation during the verification process, thereby lowering verification overhead and improving system scalability. We implemented a prototype of BAVote on both a local Ethereum network and the Sepolia testnet. Experimental results show that, with $n = 50$, our system reduces the storage cost of verification keys by more than 90% and achieves up to 7x faster verification compared with representative existing schemes, demonstrating its practicality and scalability in blockchain environments. Due to the trade-off between efficiency and decentralization, we instantiate the tracer and combiner as single entities in our system. How to relax the trust assumptions on the tracer and combiner, and achieve full decentralization can be one of future directions. In this paper, we focus on the deployment of the voting system in a blockchain environment, broader deployment scenarios such as decentralized decision-making in DAOs, and large-scale collaborative authorization, remain an important direction for future work.

REFERENCES

[1] W. Xue, Y. Yang, Y. Li, H. H. Pang, and R. H. Deng, "ACB-vote: Efficient, flexible, and privacy-preserving blockchain-based score voting with anonymously convertible ballots," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 3720–3734, 2023.

[2] J. Huang, D. He, Y. Chen, M. K. Khan, and M. Luo, "A blockchain-based self-tallying voting protocol with maximum voter privacy," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 5, pp. 3808–3820, Sep. 2022.

[3] N. Van Saberhagen, "Cryptonote V 2.0," 2013. [Online]. Available: https://cryptonote.org/whitepaper.pdf

[4] D. L. Chaum, "Untraceable electronic mail, return addresses, and digital pseudonyms," *Commun. ACM*, vol. 24, no. 2, pp. 84–88, Feb. 1981.

[5] M. Kumar, C. P. Katti, and P. Saxena, "A secure anonymous E-voting system using identity-based blind signature scheme," in *Proc. ICISS*, 2017, pp. 29–49.

[6] Y. Wu and S. Kasahara, "Smart contract-based e-voting system using homomorphic encryption and zero-knowledge proof," in *Proc. ACNS*, vol. 13907. Cham, Switzerland: Springer, 2023, pp. 67–83.

[7] D. Boneh and C. Komlo, "Threshold signatures with private accountability," in *CRYPTO 2022*, vol. 13510. Cham, Switzerland: Springer, 2022, pp. 551–581.

[8] P. McCorry, S. F. Shahandashti, and F. Hao, "A smart contract for boardroom voting with maximum voter privacy," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.* Cham, Switzerland: Springer, 2017, pp. 357–375.

[9] Y. Li et al., "A blockchain-based self-tallying voting protocol in decentralized IoT," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 119–130, Jan. 2022.

[10] Y. Yang, Z. Guan, Z. Wan, J. Weng, H. H. Pang, and R. H. Deng, "PriScore: Blockchain-based self-tallying election system supporting score voting," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 4705–4720, 2021.

[11] Y. Liu, D. He, M. Luo, L. Wang, and C. Peng, "*k*-TEVS: A *k*-times e-voting scheme on blockchain with supervision," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 3, pp. 2326–2337, Mar. 2024.

[12] D. Chaum, "Blind signature system," in *CRYPTO 1983*. New York, NY, USA: Plenum Press, 1983, p. 153.

[13] F. Zhang and K. Kim, "ID-based blind signature and ring signature from pairings," in *ASIACRYPT 2002*, vol. 2501. Cham, Switzerland: Springer, 2002, pp. 533–547.

[14] M. Hirt and K. Sako, "Efficient receipt-free voting based on homomorphic encryption," in *EUROCRYPT 2000*, vol. 1807. Cham, Switzerland: Springer, 2000, pp. 539–556.

[15] J. C. P. Carcía, A. Benslimane, and S. Boutalbi, "Blockchain-based system for e-voting using blind signature protocol," in *Proc. IEEE Global Commun. Conf., (GLOBECOM)*, Madrid, Spain, Jun. 2021, pp. 1–6.

[16] B. Yu et al., "Platform-independent secure blockchain-based voting system," in *Proc. Int. Conf. Inf. Secur.* Cham, Switzerland: Springer, 2018, pp. 369–386.

[17] M. Li et al., "S^3S3Voting: A blockchain sharding based E-voting approach with security and scalability," *IEEE Trans. Dependable Secur. Comput.*, vol. 22, no. 2, pp. 1596–1611, Feb. 2025.

[18] H. Li, Y. Li, Y. Yu, B. Wang, and K. Chen, "A blockchain-based traceable self-tallying E-voting protocol in AI era," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1019–1032, Apr. 2021.

[19] M. Miao, L. Tang, J. Li, and X. Zhang, "New blockchain-based publicly traceable self-tallying voting protocol," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 2, pp. 2034–2046, Mar. 2024.

[20] P. Li and J. Lai, "Lat-voting: Traceable anonymous e-voting on blockchain," in *NSS 2019*, vol. 11928. Cham, Switzerland: Springer, 2019, pp. 234–254.

[21] Y. Desmedt and Y. Frankel, "Threshold cryptosystems," in *CRYPTO 1989*, vol. 435. Cham, Switzerland: Springer, 1989, pp. 307–315, doi: 10.1007/0-387-34805-0.

[22] A. Boldyreva, "Threshold signatures, multisignatures and blind signatures based on the gap-Diffie–Hellman-group signature scheme," in *Proc. PKC*, vol. 2567, 2003, pp. 31–46.

[23] P. Fouque and J. Stern, "Fully distributed threshold RSA under standard assumptions," in *ASIACRYPT 2001*, vol. 2248. Cham, Switzerland: Springer, 2001, pp. 310–330.

[24] S. Micali, K. Ohta, and L. Reyzin, "Accountable-subgroup multisignatures: Extended abstract," in *Proc. 8th ACM Conf. Comput. Commun. Secur.*, Nov. 2001, pp. 245–254.

[25] J. Nick, T. Ruffing, and Y. Seurin, "MuSig2: Simple two-round Schnorr multi-signatures," in *CRYPTO 2021*, vol. 12825. Cham, Switzerland: Springer, 2021, pp. 189–221.

[26] S. Das, P. Camacho, Z. Xiang, J. Nieto, B. Bünz, and L. Ren, "Threshold signatures from inner product argument: Succinct, weighted, and multi-threshold," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Copenhagen, Denmark, Nov. 2023, pp. 356–370.

[27] S. Garg, A. Jain, P. Mukherjee, R. Sinha, M. Wang, and Y. Zhang, "HinTS: Threshold signatures with silent setup," in *Proc. IEEE Symp. Secur. Privacy (SP)*, San Francisco, CA, USA, May 2024, pp. 3034–3052.
[28] D. Boneh, A. Partap, and B. Waters, "Accountable multi-signatures with constant size public keys," in *Proc. PKC* (LNCS). Springer, 2025, pp. 32–65.

Ziyi Lin is currently pursuing the master's degree with the School of Cyberspace Science and Technology, Beijing Institute of Technology. Her research interests include threshold signatures and blockchain technology.

Peng Jiang received the Ph.D. degree from Beijing University of Posts and Telecommunications in 2017. She is currently an Associate Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing. She has edited three books/book chapters and published more than 40 refereed papers in international journals and conferences. Her research interests include cryptography, information security, and blockchain technology. She serves as the Co-Chair of SIG for the IEEE TEMS's TC on Blockchain and Distributed Ledger Technologies. She has served as an Associated Editor for *Computer Standards and Interfaces*, a guest editor for many journals, and a program committee member for many conferences.

Zijian Zhang (Senior Member, IEEE) received the Ph.D. degree from the School of Computer Science and Technology, Beijing Institute of Technology. He was a Visiting Scholar with the Computer Science and Engineering Department, State University of New York at Buffalo, in 2015. He is currently a Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology. His research interests include the design of authentication and key agreement protocols and the analysis of entity behavior and preference.

Liehuang Zhu (Senior Member, IEEE) received the Ph.D. degree in computer science from Beijing Institute of Technology. He is currently a Professor and the Dean of the School of Cyberspace Science and Technology, Beijing Institute of Technology. He has published more than 100 peer-reviewed journals or conference papers, including more than ten IEEE/ACM TRANSACTIONS articles. His research interests include security protocol analysis and design, wireless sensor networks, and cloud computing. He has been granted several IEEE best paper awards, including IWQoS'17 and TrustCom'18.